

Examen : Visualisation des séries temporelles avec Bokeh

Cet examen teste votre capacité à visualiser des données de séries temporelles en utilisant la bibliothèque Bokeh. Vous travaillerez avec le jeu de données « Températures minimales quotidiennes à Melbourne ». Pour chaque question, fournissez le code Python en utilisant Bokeh pour générer la visualisation demandée.

Ensemble de données : « daily-minimum-temperatures-in-melbourne.csv »

Dans [1...

```
import pandas as pd
from bokeh.plotting import figure, show
from bokeh.io import output_notebook
from bokeh.resources import INLINE
from bokeh.models import (
    ColumnDataSource,
    HoverTool,
    DateRangeSlider
)
from bokeh.layouts import column
from bokeh.transform import factor_cmap
from bokeh.palettes import viridis
import os

# Utiliser INLINE force Bokeh à charger son code JavaScript localement
# plutôt que de le télécharger depuis internet, ce qui règle le bug de chargement infini.
output_notebook(resources=INLINE)

# Load the Dataset
df = None
source = None

try:
    filepath = "../datasets/daily-minimum-temperatures-in-melbourne.csv"
    if not os.path.exists(filepath):
        filepath = "./datasets/daily-minimum-temperatures-in-melbourne.csv"

    # Specify engine='python' or remove on_bad_lines if using older pandas
    df = pd.read_csv(filepath, on_bad_lines='skip', engine='python')

    # Rename columns for clarity
    df.columns = ['Date', 'Temperature']

    # Convert the 'Date' column to datetime format
    df['Date'] = pd.to_datetime(df['Date'])

    # Remove '?' from the 'Temperature' column and convert to numeric
    df['Temperature'] = df['Temperature'].astype(str).str.replace(r'^0-9.', '', regex=True)
    df['Temperature'] = pd.to_numeric(df['Temperature'], errors='coerce')
    df.dropna(inplace=True) # Drop rows where conversion failed

    source = ColumnDataSource(df)
    print(f"Dataset loaded and preprocessed successfully from {filepath}.")

except FileNotFoundError:
    print("Error: The dataset file was not found. Please ensure 'daily-minimum-temperatures-in-melbourne.csv' is in the 'datasets' directory.")
except Exception as e:
    print(f"An error occurred: {e}")
```



BokehJS 3.10.0-dev.2 a été chargé avec succès.

Dataset loaded and preprocessed successfully from ../datasets/daily-minimum-temperatures-in-melbourne.csv.

Question 1 : Intrigue de base des séries temporelles

1. Créez un graphique linéaire basique montrant la température minimale quotidienne au fil du temps.

- Utilisez la colonne « Date » sur l'axe des x et la colonne « Température » sur l'axe des y.
- Définissez le titre du graphique sur « Températures minimales quotidiennes ».
- Étiquetez l'axe des x comme « Date » et l'axe y comme « Température (°C) ».
- Ajoutez des bulles pour afficher la date et la température lorsque vous survolez la ligne.
- Activez les outils de panoramique, de zoom sur la molette et de réinitialisation.

Dans [2...

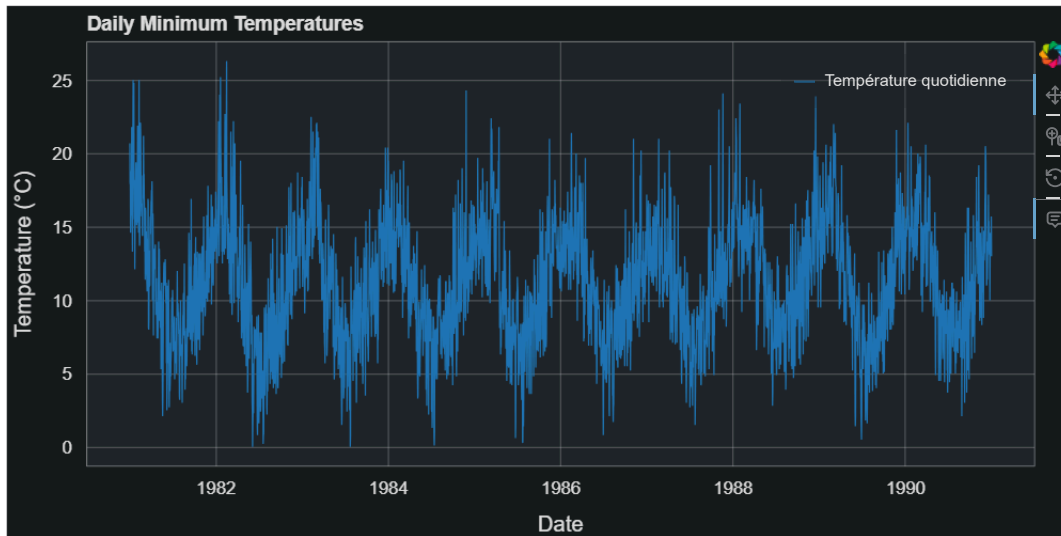
```
# Answer 1:
if source is not None:
    p1 = figure(
        title="Daily Minimum Temperatures",
        x_axis_type="datetime",
        width=800,
        height=400,
        tools="pan,wheel_zoom,reset"
    )

    p1.line(x='Date', y='Temperature', source=source, legend_label="Daily Temp")

    p1.xaxis.axis_label = "Date"
    p1.yaxis.axis_label = "Temperature (°C)"

    hover = HoverTool(
        tooltips=[("Date", "@Date{%F}"), ("Temperature", "@Temperature{0.0}°C")],
        formatters={'@Date': 'datetime'}
    )
    p1.add_tools(hover)

    show(p1)
```



Question 2 : Moyenne roulante 2. Calculer la moyenne mobile sur 30 jours de la température minimale quotidienne et la tracer En plus des données de température originales.

- * Create a new column 'Rolling_Avg' in the DataFrame containing the 30-day rolling average.
- * Plot both the original 'Temperature' and the 'Rolling_Avg' on the same plot.
- * Use different colors and line styles to distinguish between the two.
- * Add a legend to the plot to label the lines.
- * Add tooltips to display the date, original temperature, and rolling average.

Dans [3...

```
# Answer 2:
if df is not None:
    df['Rolling_Avg'] = df['Temperature'].rolling(window=30).mean()
    source_q2 = ColumnDataSource(df)

    p2 = figure(
        title="Daily Temperatures with 30-Day Rolling Average",
        x_axis_type="datetime",
        width=800,
        height=400,
        tools="pan,wheel_zoom,reset"
    )

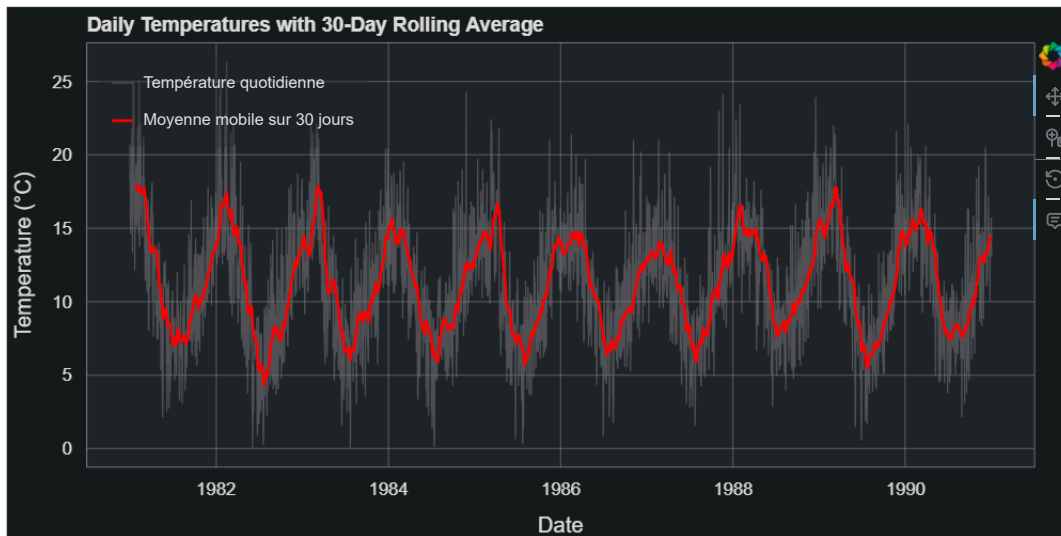
    p2.line(x='Date', y='Temperature', source=source_q2, color='gray', alpha=0.5, legend_label="Daily Temp")
    p2.line(x='Date', y='Rolling_Avg', source=source_q2, color='red', line_width=2, legend_label="30-Day Rolling Avg")

    p2.xaxis.axis_label = "Date"
    p2.yaxis.axis_label = "Temperature (°C)"
    p2.legend.location = "top_left"
    p2.legend.click_policy = "hide"

    hover = HoverTool(
        tooltips=[
            ("Date", "@Date{%F}"),
            ("Temperature", "@Temperature{0.0}°C"),
            ("Rolling Avg", "@Rolling_Avg{0.0}°C")
        ],
        formatters={'@Date': 'datetime'}
```

```
)
p2.add_tools(hover)

show(p2)
```



Question 3 : Box plots mensuels 3. Créer des diagrammes en boîte pour visualiser la répartition des températures pour chaque mois.

- * Extract the month from the 'Date' column and create a new 'Month' column.
- * Group the data by 'Month' and prepare it for plotting.
- * Use Bokeh's box plot elements to visualize the distribution.
- * Label the x-axis with month names and the y-axis with "Temperature (°C)".
- * Add tooltips to display the month and relevant statistical values (min, max, median, etc.).

Dans [4...

```
# Answer 3:
if df is not None:
    df['Month'] = df['Date'].dt.strftime('%m-%B')

    # Calculate quartiles and whiskers for box plot
    q1 = df.groupby('Month')['Temperature'].quantile(q=0.25)
    q2 = df.groupby('Month')['Temperature'].quantile(q=0.5)
    q3 = df.groupby('Month')['Temperature'].quantile(q=0.75)
    iqr = q3 - q1
    upper = q3 + 1.5 * iqr
    lower = q1 - 1.5 * iqr

    outliers = df[(df['Temperature'] > upper[df['Month']].values) | (df['Temperature'] < lower[df['Month']].values)]
    months = sorted(df['Month'].unique())

    p3 = figure(
        title="Monthly Temperature Distribution",
        x_range=months,
        width=800,
        height=400,
        tools="pan,wheel_zoom,reset"
    )
```

```

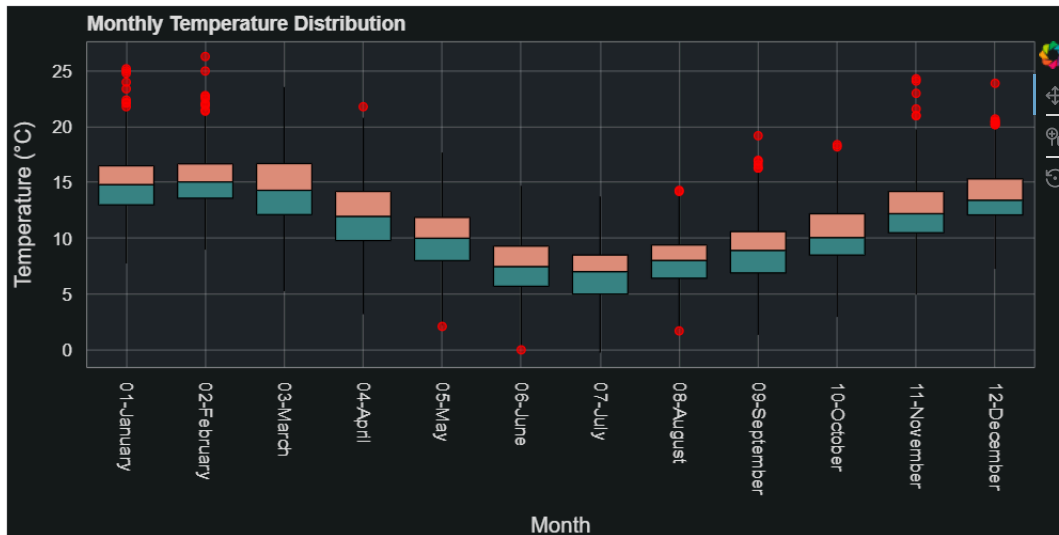
p3.segment(months, upper, months, q3, line_color="black")
p3.segment(months, lower, months, q1, line_color="black")
p3.vbar(months, 0.7, q2, q3, fill_color="#E08E79", line_color="black")
p3.vbar(months, 0.7, q1, q2, fill_color="#3B8686", line_color="black")

if not outliers.empty:
    p3.scatter(x=outliers['Month'], y=outliers['Temperature'], size=6, color="red", fill_alpha=0.6)

p3.xaxis.axis_label = "Month"
p3.yaxis.axis_label = "Temperature (°C)"
p3.xaxis.major_label_orientation = "vertical"

show(p3)

```



Question 4 : Diagrammes annuels avec mappage des couleurs 4. Créer des diagrammes en boîte pour visualiser la répartition des températures pour chaque année, et utiliser la cartographie des couleurs pour mettre en évidence les variations de température.

- * Extract the year from the 'Date' column and create a new 'Year' column.
- * Group the data by 'Year' and prepare it for plotting.
- * Use Bokeh's box plot elements to visualize the distribution for each year.
- * Label the x-axis with the 'Year' and the y-axis with "Temperature (°C)".
- * Use 'factor_cmap' to color the boxes based on the median temperature of each year.
- * Add tooltips to display the year and relevant statistical values (min, max, median, etc.).
- * Enable pan, wheel zoom, and reset tools.

Dans [5...

```

# Answer 4:
if df is not None:
    df['Year'] = df['Date'].dt.year

    q1_yr = df.groupby('Year')['Temperature'].quantile(q=0.25)
    q2_yr = df.groupby('Year')['Temperature'].quantile(q=0.5)
    q3_yr = df.groupby('Year')['Temperature'].quantile(q=0.75)

    df_stats_yr = pd.DataFrame({'q1': q1_yr, 'q2': q2_yr, 'q3': q3_yr}).reset_index()
    source_q4 = ColumnDataSource(df_stats_yr)
    years_str = [str(y) for y in df_stats_yr['Year']]

```

```

num_years = len(years_str)
palette = viridis(num_years)

p4 = figure(
    title="Yearly Temperature Distribution",
    x_range=years_str,
    width=900,
    height=450,
    tools="pan,wheel_zoom,reset"
)

color_map = factor_cmap('Year', palette=palette, factors=years_str)

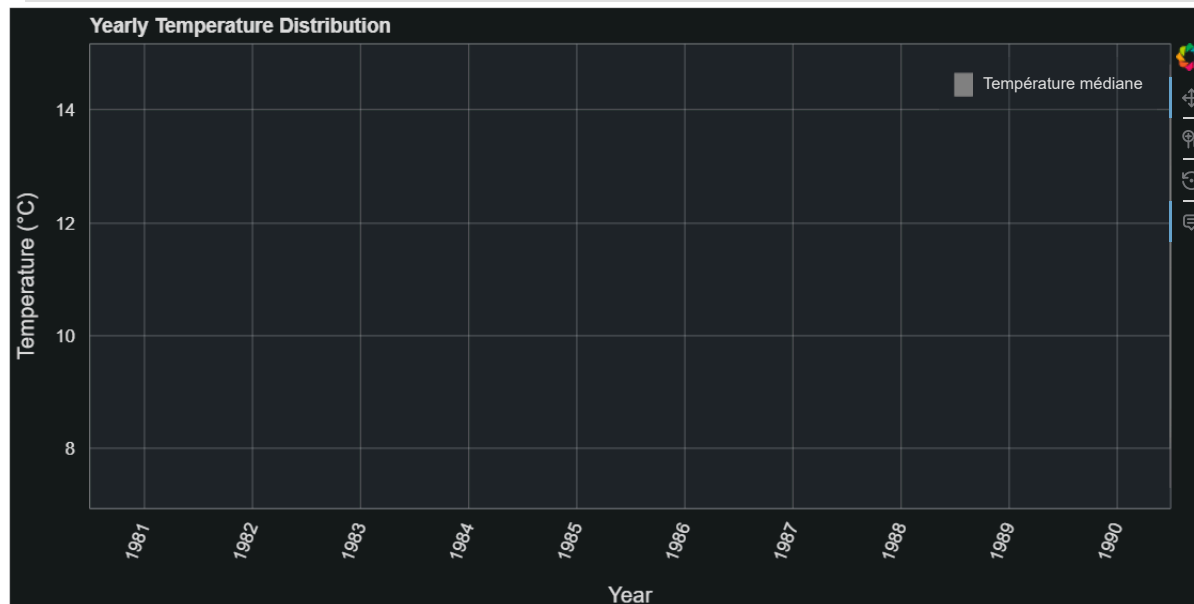
p4.vbar(x='Year', top='q3', bottom='q2', width=0.8, source=source_q4, color=color_map, legend_label="Median Temp")
p4.vbar(x='Year', top='q2', bottom='q1', width=0.8, source=source_q4, color=color_map)

p4.xaxis.axis_label = "Year"
p4.yaxis.axis_label = "Temperature (°C)"
p4.xaxis.major_label_orientation = 1.2

hover = HoverTool(tooltips=[
    ("Year", "@Year"),
    ("Median", "@q2{0.2f}°C"),
    ("Q1", "@q1{0.2f}°C"),
    ("Q3", "@q3{0.2f}°C"),
])
p4.add_tools(hover)

show(p4)

```



Question 5 : Sélection interactive de plage temporelle

5. Créez un graphique de lignes interactif où l'utilisateur peut sélectionner une plage de temps spécifique pour visualiser en utilisant un curseur de plage de dates.

- Créez un graphique linéaire basique de « Température » sur « Date ».
- Implémentez un curseur de plage de dates à l'aide de widgets Bokeh pour permettre aux utilisateurs de sélectionner une date de début et de fin.
- Mettez à jour le graphique dynamiquement en fonction de la plage de dates sélectionnée.
- Ajoutez des infobulles pour afficher la date et la température.
- Activez les outils de panoramique, de zoom sur la molette et de réinitialisation.

Dans [6...

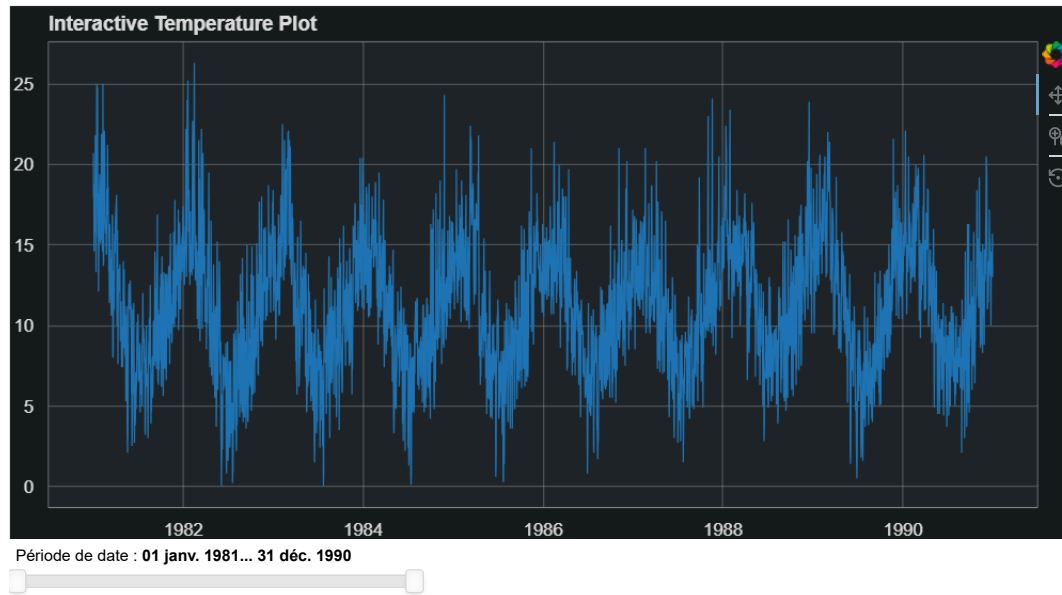
```
# Answer 5:
if source is not None and df is not None:
    p5 = figure(
        title="Interactive Temperature Plot",
        x_axis_type="datetime",
        width=800,
        height=400,
        tools="pan,wheel_zoom,reset"
    )
    p5.line(x='Date', y='Temperature', source=source)

    date_range_slider = DateRangeSlider(
        title="Date Range",
        start=df['Date'].min(),
        end=df['Date'].max(),
        value=(df['Date'].min(), df['Date'].max()),
        step=1
    )

    from bokeh.models import CustomJS
    callback = CustomJS(args=dict(x_range=p5.x_range), code="""
        const start = cb_obj.value[0];
        const end = cb_obj.value[1];
        x_range.start = start;
        x_range.end = end;
    """)

    date_range_slider.js_on_change('value', callback)

    layout = column(p5, date_range_slider)
    show(layout)
```



Question 6 : Visualisation de la décomposition des séries temporelles

6. Effectuez une simple décomposition de séries temporelles pour visualiser la tendance et la saisonnalité composantes des données de température.

- Rééchantillonnez les données en fréquence mensuelle et calculez la température moyenne mensuelle.
- Utilisez une moyenne mobile simple pour estimer la composante de tendance.
- Calculez la composante saisonnière en soustrayant la tendance des données mensuelles initiales.
- Créer trois diagrammes Bokeh distincts : un pour les données mensuelles originales, un pour la tendance, et un pour la composante saisonnière.
- Assurez-vous que les graphiques sont alignés et partagent le même axe des x (Date).
- Ajoutez des infobulles à chaque graphique pour afficher la date et la valeur correspondante.
- Activez les outils de panoramique, de zoom en roue et de réinitialisation pour chaque parcelle.

Dans [7...

```
# Answer 6:
if df is not None:
    monthly_df = df.set_index('Date')[['Temperature']].resample('ME').mean()

    # Trend: 12-month moving average
    monthly_df['Trend'] = monthly_df['Temperature'].rolling(window=12, center=True).mean()

    # Seasonality
    monthly_df['Seasonal'] = monthly_df['Temperature'] - monthly_df['Trend']

    source_q6 = ColumnDataSource(monthly_df)

    # Plot 1: Original Monthly Data
    p6_orig = figure(title="Original Monthly Average Temperature", x_axis_type='datetime', width=800, height=250, tools="pan,wheel_zoom,reset")
    p6_orig.line(x='Date', y='Temperature', source=source_q6, legend_label="Monthly Avg")

    # Plot 2: Trend
    p6_trend = figure(title="Trend Component", x_axis_type='datetime', width=800, height=250, x_range=p6_orig.x_range, tools="pan,wheel_zoom,reset")
    p6_trend.line(x='Date', y='Trend', source=source_q6, color='red', legend_label="Trend (12-mo MA)")
```



```
# Plot 3: Seasonal
p6_seasonal = figure(title="Seasonal Component", x_axis_type='datetime', width=800, height=250, x_range=p6_orig.x_range, tools="pan,wheel_zoom,reset")
p6_seasonal.line(x='Date', y='Seasonal', source=source_q6, color='green', legend_label="Seasonal")

layout = column(p6_orig, p6_trend, p6_seasonal)
show(layout)
```

